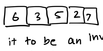
Count Inversions



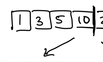
for it to be an inversion pair
i < j
arr[i] > arr[j]

6 > 3 → 1
6 > 5 → 1
6 > 2 → 1
3 > 2 → 1
5 > 2 → 1
5

⇒ 5 is the count that needs to be returned

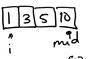
Brute force : run two loops & count TC : O(n²)

optimal : Merge Sort

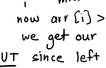


* here both the halfs are already sorted, if they weren't then just continue merge sort (get LC & RC from that)

*i < j will always be true



arr[i] < arr[j]
just store in temp
no inv



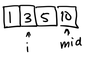
now arr[i] > arr[j]
we get our inv pair

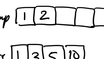
BUT since left side is sorted values after 3 are greater than 3 ie they're greater than 2 which satisfies the inv condition
ie 3 > 2 , 5 > 2 , 10 > 2

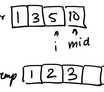
SO basically we will directly calculate the count instead of iterating further

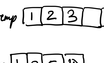
* `invCount += mid - i + 1`

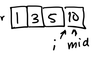
here mid = 3 i = 1
3 - 1 + 1 = 3
we get 3 pairs

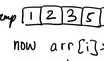




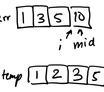


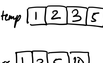


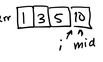


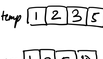


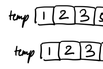
now arr[i] > arr[j]
we get one pair
in formula
invCount += mid - i + 1
0
so essentially we are incrementing by 1

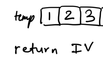












return IV

code :

```
int mergeSort(arr, st, end) {
  if (st < end) {
    mid = st + (end - st)/2
    LC = mergeSort(arr, st, mid)
    RC = mergeSort(arr, mid+1, end)
    IVC = merge(arr, st, mid, end)
    return IVC + LC + RC;
  }
  return 0;
}
int merge(arr, st, mid, end) {
  vector<int> temp
  i = st , j = mid+1
  while (i <= mid && j <= end) {
    if (arr[i] <= arr[j]) {
      temp.pb(arr[i])
      i++
    } else {
      temp.pb(arr[j])
      j++
      invCount += mid - i + 1
    }
  }
  while (i <= mid) {
    temp.pb(arr[i])
    i++
  }
  while (j <= end) {
    temp.pb(arr[j])
    j++
  }
  for (idx = 0 ; idx < temp.size() ; idx++) {
    arr[idx + st] = temp[idx];
  }
  return invCount
}
```